ENGR 102 Summer Bridge
Bridge Day 12 Student Workbook

For Loops, Range, Counters, and Accumulators

Tuesday, July 21, 2026 • 50 points • tagged v5 successor

Name		**UIN**		**Section**	

Daily target and independence boundary

Build fixed-count programs that preserve loop state and exact bounds.

Allowed tools	Day 10 tools plus for, range, counters, and accumulators
Support supplied	Required inputs, required outputs, and one sample run are supplied.
Student decides	Initialization, range bounds, accumulator updates, and the placement of the threshold decision.
Scaffold level	State inventory prompted once; loop body is student-authored.

Evidence and scoring

Evidence	**Points**	**Secure work shows**
Integrated trace	10	intermediate state, condition results, exact output, and meaning
Diagnosis and repair	10	actual, intended, cause, repair, and a discriminating test
Complete program and tests	30	coherent executable logic, required output, assumptions, and defended tests

Task 1. Integrated trace - 10 points

Trace the range, the conditional counter, and the accumulator after every pass.

```
total = 0
multiple_count = 0

for number in range(2, 11, 2):
    if number % 4 == 0:
        multiple_count = multiple_count + 1
    total = total + number - multiple_count
    print(number, total, multiple_count)
```

Your task

1. List the values produced by range(2, 11, 2).
2. Create one pass ledger containing number, multiple_count after the if, and total after the update.
3. Write all printed lines in exact order.

Task 2. Diagnose and repair - 10 points

The intended program must add the integers from 1 through limit, inclusive.

```
limit = int(input())
total = 0

for number in range(1, limit):
    total = total + number

print(total)
```

Your task

1. For limit = 4, state the actual and intended totals and identify the exact cause.
2. Write the minimal repair and a boundary test that would expose the original bug.

Task 3. Construct a complete program - 30 points

Program specification

- Read sample_count as an integer and threshold as a float.
- If sample_count is not positive, print Invalid count and do not ask for readings.
- Otherwise read exactly sample_count floating-point readings.
- Compute the total, average, and number of readings at or above threshold.
- Print total, average, and at_or_above on separate lines in that order.

Required planning evidence

1. Name the state that must exist before the loop and give each initial value.

Program workspace

Task 3 continued. Test and defend - included in 30 points

Continue code if needed. Required tests, expected results, and brief defense

Bridge Day 12 VIP Evidence Handoff

STUDENT COPY • Contract 07a04826c975 • Accessible v5 successor

Why engineers use this page

Use deliberate range bounds and comparison state to collect multiples and preserve the smallest value.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: for loop, while loop, range call, loop state update, termination condition, list literal as collector, append call
Carried authoring tools: assignment, print call, arithmetic expression, modulo, input call, int conversion, f string, comparison expression, if elif else
Supplied-code exposure only: list index read
An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.18	Multiples In Range	`18-18-work / 18-18-transfer / 18-18-cold`	append call, arithmetic expression, for loop, range call
18.20	Smallest Value	`18-20-work / 18-20-transfer / 18-20-cold`	comparison expression, for loop, if elif else

Readiness evidence

1. Write the first value, final included value, and excluded stop value for the range used in Activity 18.18.

2. Explain why Activity 18.20 initializes from real data instead of from zero.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	
Observed Result	
Revision Reason	
Engineering Meaning	
Units Or Not Applicable	
Assumption	
Model Limit	
Next Test	
Help Trigger	
Minutes Used	

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Generated from the canonical VIP construct and evidence contract; workbook planning and executable notebook work remain distinct.